INTI International College Penang **School of Computing**

3+0 Bachelor of Science (Hons) in Computer Science, in collaboration with Coventry University, UK

3+0 Bachelor of Science (Hons) in Computing, in collaboration with Coventry University, UK

Coursework cover sheet

Section A - To be completed by the student

Full Name: Leong Zi Qi, Boon Yun Nee, Pey Hui Yi, Tan Zhi Qi	
CU Student ID Number: 16290514, 16290411, 16290569, 16290639	
Semester: April 2026	
Lecturer: Mr. Shahriman Mohd Said	
Module Code and Title: 6005MD Web API Development	
Assignment No. / Title: Smart Travel Planner: Integrating Self-Developed and Third-Party APIs	% of Module Mark: 50%
Hand out date: 14/4/2026	Due date: 15/6/2026, 23.59pm Malaysian Time
Penalties: No late work will be accepted. If you are unable to submit coursework on time due to extenuating circumstances you may be eligible for an extension. Please consult the lecturer.	
Declaration: I/we the undersigned confirm that I/we have read and agree to abide by the University regulations on plagiarism and cheating and Faculty coursework policies and procedures. I/we confirm that this piece of work is my/our own. I/we consent to appropriate storage of our work for plagiarism checking.	

Section B - To be completed by the module leader

Learning Outcomes 2. Consider the process of constructing a simple Web API by defining its scope, designing its architecture, and implementing it to solve a given problem. (C5, P2) 3. Display the ability to implement Web API security and authentication features. (P5, P3)
Lecturer's Feedback
Internal Moderator's Feedback

Table of Contents

System Architecture..... 1
Database Design & Integration..... 1
API Description & Endpoints..... 3
API Starting Point..... 5
API Aggregation Architecture..... 6
Recommendation-based API..... 8
Weather-based API..... 14
Caching and Performance Optimisation..... 20
Map/Location-based API..... 21
Teammate Individual Portfolio..... 29
Security & Centralized Error Handling..... 31
Testing Framework & Test Results..... 33
Challenges, Limitations & AI Reflection..... 34
Links..... 34
References..... 35
Appendix..... 38

System Architecture

The Smart Travel Planner utilizes a highly resilient, decoupled, distributed client-server architecture. The backend operates as a secure Node.js/Express REST API middleware gateway proxy, which effectively shields the client application from directly calling external APIs (Node.js Foundation, n.d.). The application layer features a cross-platform client built in the Flutter framework using the Dart programming language (Google, n.d.-a), which communicates with the backend via an automated token-injection HTTP client.

Our group structured the API routing into clean, isolated modules under a unified /api base path. This is orchestrated by a central route registry (src/routes/index.js) that mounts specific feature routers:

- /api/auth: Handles user session management and account registration.
- /api/trips: Controls user travel records and historical logs stored in the database.
- /api/external: Serves as a gateway proxy for real-time external data streams.
- /api/countries: Exposes parsed geo-demographic data.

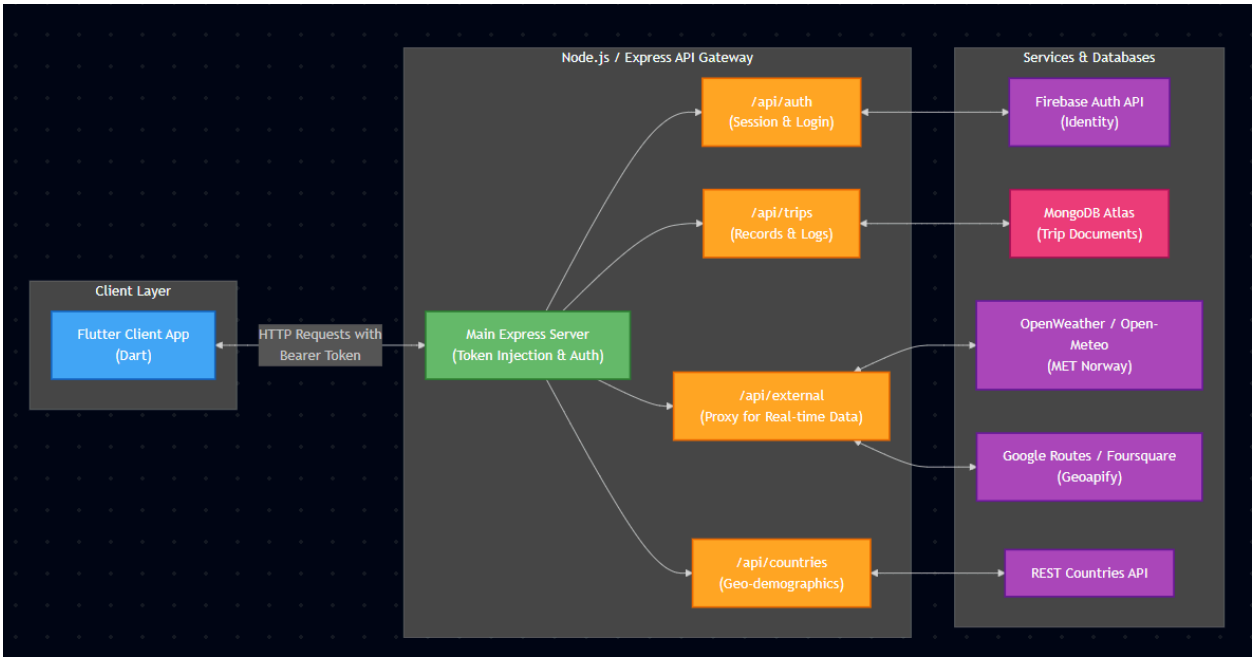


Figure 1: Full-stack system architecture flow.

Database Design & Integration

For primary data storage, the system integrates a MongoDB Atlas Cloud Database cluster. (MongoDB, n.d.). This is managed via a centralized configuration utility (src/db/mongo.js), where connection handles are safely initialized at runtime using the environment variables (MONGO_URI and MONGO_DB_NAME).

To elevate the system's security score to the highest rubric bracket, our group implemented a dual database and authentication strategy:

1. **Identity Management:** Managed via the Firebase Auth REST API / Identity Toolkit. (Google, n.d.-b). User registration and sign-in requests pass through backend endpoints to target Firebase endpoints (accounts:signUp, accounts:signInWithPassword, accounts:lookup).
2. **Application Records:** Trip logs, agendas, and travel notes are tied to the validated Firebase User ID and maintained within isolated MongoDB document collections (MongoDB, n.d.).
3. **External API Caching**: External APIs are expensive and free tiers have strict limitations. To ensure that the system is not exhausted by repeated operations, caching the retrieved external API data is extremely important to reduce redundant calls.

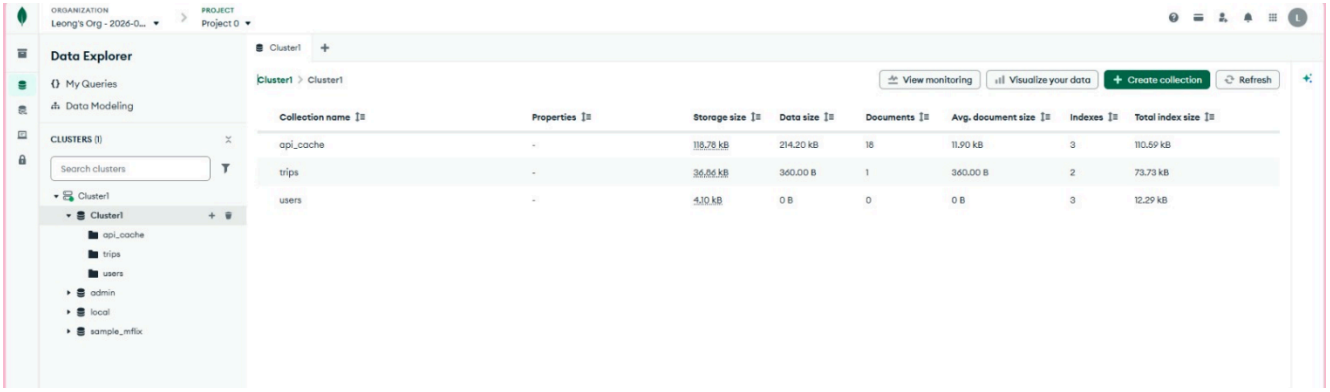


Figure 2: MongoDB document schema for travel records.

MONGODB_USER			
PK	id	string	Auto-generated ID (e.g. user_001)
	firebaseLocalId	string	Linked Firebase Identity Toolkit UID
	email	string	User's registered email
	name	string	User's display name
	emailVerified	boolean	Verification status

M

M

MONGODB_TRIP			
PK	id	string	Auto-generated ID (e.g. trip_001)
FK	ownerUserId	string	Links to MONGODB_USER.id
	destinationName	string	e.g., Tokyo
	destinationCountry	string	e.g., Japan
	latitude	number	Geo-coordinate
	longitude	number	Geo-coordinate
	startDate	date	Start of travel
	endDate	date	End of travel
	preferences	array	User's POI preferences (e.g. 'restaurants')
	travelNotes	string	User's personal text notes

Figure 3: Database model mapping users to trip summary

API Description & Endpoints

The system complies with REST standards without deviation. The following is our actual route registry configuration linking our custom developed database to several third party data sources including OpenWeather, Google Maps, Foursquare, REST Countries (Current weather data, n.d.; Florez, F. n.d.; Foursquare, n.d.; Google, n.d.; OpenWeather, n.d.).

HTTP Method	Route Endpoint Path	Service File Location	Description / Purpose
GET	/api/health	src/routes/index.js	Heartbeat endpoint to monitor backend uptime status.
POST	/api/auth/register	src/routes/auth.routes.js	Submits data to Firebase Auth REST to create user records.
POST	/api/auth/login	src/routes/auth.routes.js	Authorizes users and returns secure session bearer tokens.
GET	/api/auth/profile	src/routes/auth.routes.js	Retrieves active user metadata and profile options.
POST	/api/trips	src/routes/trips.routes.js	Appends a new travel record, preferences, and destination to MongoDB.

GET	/api/trips	src/routes/trips.routes.js	Queries the collection to fetch all trips assigned to the logged-in user.
PUT	/api/trips/:id	src/routes/trips.routes.js	Modifies fields, notes, or agenda items for a specific trip.
DELETE	/api/trips/:id	src/routes/trips.routes.js	Purges a specific trip document from the database by unique ID.
GET	/api/external/weather	src/services/weatherService.js	Proxy route fetching real-time weather alerts and conditions.
GET	/api/external/route	src/services/routeService.js	Computes driving and walking navigation paths using coordinates.
GET	/api/external/recommendations	src/services/foursquareService.js	Queries localized point-of-interest suggestions based on preferences.
GET	/api/countries/:name	src/services/countryService.js	Fetches standalone capital, flag, and currency demographic metadata.
GET	/api/trips/:id/summary	src/routes/summaryRoutes.js	Master Aggregation Endpoint: Compiles data from all modules.

API Starting Point



```
/**
 * Creates the app data.
 */
Windsurf: Refactor | Explain | ×
function createApp() {
  const app = express();

  app.use(
    cors({
      origin: resolveCorsOrigin,
      credentials: true,
    }),
  );

  app.use(express.json({ limit: "1mb" }));
  app.use(express.urlencoded({ extended: false }));

  app.get("/", (req, res) => {
    res.json({
      success: true,
      message: "Smart Travel Planner API is running.",
      apiBase: "/api",
    });
  });

  app.use("/api", apiRouter);

  return app;
}
```

Figure 4: Smart Travel Planner API Setup

This is the main Express.js application setup for the Smart Travel Planner API. The createApp() function is used to create the Express server application. After that, the first line in the function will be used to initialize a new Express instance. The **CORS** is required to be enabled to prevent browsers from blocking frontend to the backend requests. Besides that, the JSON middleware is also limited to 1mb to reduce risk of large payloads to throttle the system performance. Furthermore, the function also defines a root route that returns a JSON response confirming that the Smart Travel Planner API is running. The main API router is also mounted under the **'/api'** path, this ensures that all application endpoints are grouped under a consistent API base route. So, this function acts as the central setup point for the backend server and provides a structured foundation for handling authentication, trip planning, recommendations, weather services and also agenda planner.

API Aggregation Architecture

The Smart Travel Planner backend acts as an API aggregation layer between the Flutter application and multiple external data providers. The Flutter application does not need to understand each provider's authentication method, URL structure, response format, rate limit, or failure behavior. Instead, it calls the project's own Express API under `/api` and receives stable, application-specific JSON.

The backend performs five main responsibilities:

- Accept and validate a request from Flutter (The UI/UX)
- Load local trip and user data from MongoDB (The database)
- Read cached data before consuming an external provider
- Call one or more external APIs, apply fallback rules, and normalize their responses.
- Combine the normalized data into one response owned by this project.

The API router is mounted at '/api' path, then the examples of features routers will be '/trips', '/external', '/countries', '/auth' and '/health' endpoints.

There are 2 aggregation entry styles, which are direct external proxy endpoints and also trip-aware endpoints. The direct external proxy endpoints under '/api/external' will accept provider inputs directly, for example:

- `GET /api/external/weather?lat=...&lng=...`
- `GET /api/external/weather/forecast?lat=...&lng=...&startDate=...&endDate=...`
- `GET /api/external/recommendations?lat=...&lng=...&preferences=food,culture`
- `GET /api/external/route?fromLat=...&fromLng=...&toLat=...&toLng=...&mode=walk`

Besides that, the trip-aware endpoints under `/api/trips/:id` will first load the JWT authenticated user's trip. For example of `GET /api/trips/:id/weather`, the JWT will be authenticated, then it will load the trip id and user id only proceed to read the 'latitude' and 'longitude' from the trip. It will first check the trip weather's cache from the database, if there is no data or outdated, the weather provider's API should be called. Lastly, return the weather data and also cache the metadata into the database.

This approach helps to prevent repeated sending the same destination data and also ensures that users can only aggregate data for trips they own, not viewing other users also.

The main aggregation endpoint is

```
181   const summary = {
182     trip: responseTrip,
183     weather,
184     dailyWeatherForecast,
185     googlePlaces,
186     recommendations,
187     recommendationGroups,
188     recommendationLimit,
189     recommendationLimits: Object.fromEntries(
190       recommendationGroups.map((group) => [group.preference, group.limit]),
191     ),
192     countryInfo,
193     tripAgenda,
194     agenda: tripAgenda,
195   };
196   await cacheRepository.setCachedValue("trip_summary", cacheKey, summary, {
197     metadata: { tripId },
198   });
199   return summary;
```

Figure 5: Aggregation of all APIs data into a Summary

Hence, the final aggregation shape will be as provided below:

{"trip": {}, "weather": {},"dailyWeatherForecast": {},"googlePlaces": [], "recommendations": [], "recommendationGroups": [],"recommendationLimit": 5, "recommendationLimits": {},"countryInfo": {}, "tripAgenda": {}, "agenda": {}}

The value is then written to MongoDB under the ‘trip_summary’ namespace then returns a success envelope.

Recommendation-based API

The Foursquare API allows developers or business partners to search for venues, display lists or maps of locations and generate smart location recommendations based on popularity and user sentiment. (Smith, n.d.)

In the Smart Travel Planner system, Foursquare was actually initially intended to be the main external service for place retrieving and venue recommendations. This API will help in providing nearby places such as restaurants, tourist attractions, local services and much more based on the user preferences. This greatly improves the system because users are able to explore nearby facilities and activities that match their travel purposes. Hence, the application becomes more interactive and practical compared to a static travel itinerary, as users are able to view suggested locations dynamically based on their destination.

However, relying completely on a third-party API also introduces several limitations, such as request limits, network dependency and possible service unavailability. So, when Foursquare is not working due to unavailability, there will be a fallback to call the Geoapify API, where the response data can also be mapped into the same recommendation fields.

Foursquare API Integration

The system was connected with the Foursquare API to retrieve places data. The foursquareService.js file defines and configures two API endpoints.

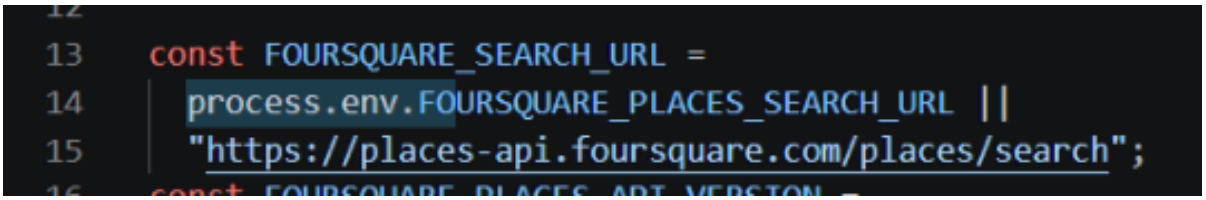


Figure 6: Foursquare API endpoint configuration for recommendation services

This Foursquare endpoint is used to retrieve real-time recommendations based on the provided preferences and radius, while the endpoint provides actual places data for the specific place, including the precise location, link, distance and several more data. The application sends requests using the destination's latitude, longitude, preferences, radius and limit count to retrieve the response from Foursquare.

For example, when there is a need to request for foursquare recommendations data, the following parameters are included:



```
Windsurf: Refactor | Explain | ×
async function requestFoursquare({
  apiKey,
  latitude,
  longitude,
  preference,
  limit,
  openAt,
  authMode,
}) {
  const url = new URL(FOURSQUARE_SEARCH_URL);
  url.searchParams.set("ll", `${latitude},${longitude}`);
  url.searchParams.set("query", queryForPreference(preference));
  url.searchParams.set("radius", DEFAULT_RADIUS_METERS);
  url.searchParams.set("limit", limit);
  url.searchParams.set("sort", "distance");
  if (openAt) {
    url.searchParams.set("open_at", openAt);
  }
  url.searchParams.set(
    "fields",
    [
      "fsq_place_id",
      "name",
      "categories",
      "distance",
      "latitude",
      "longitude",
      "location",
      "hours",
      "hours_popular",
      "link",
      "website",
    ].join(","),
  );
```

Figure 7: Configuration of Foursquare API request parameters using URLSearchParams

The requestFoursquare() function is to construct a request to the Foursquare API based on the user's selected location and travel preference. It receives latitude, longitude, preference, result limit and optional opening time as input parameters. The function then appends these values as query parameters to the Foursquare search URL. The location is passed using latitude and longitude, while the user's preference will be converted into a suitable search keyword. The request also includes a search radius, result limit and distance-based sorting so that nearby places are prioritised. In addition, the function specifies selected response fields such as place name, category, distance, coordinates, location details, opening hours, links and website information. This allows the Smart Travel Planner system to retrieve only the relevant venue data needed for displaying place recommendations to users.

Geoapify API Integration

When the Foursquare API is unavailable, it will fallback to the Geoapify API for recommendations data. The geoapifyService.js file defines and configures two API endpoints.

```
13  const GEOAPIFY_PLACES_URL =
14    process.env.GEOAPIFY_PLACES_URL || "https://api.geoapify.com/v2/places";
```

Figure 8: Geoapify API endpoint configuration for recommendation services

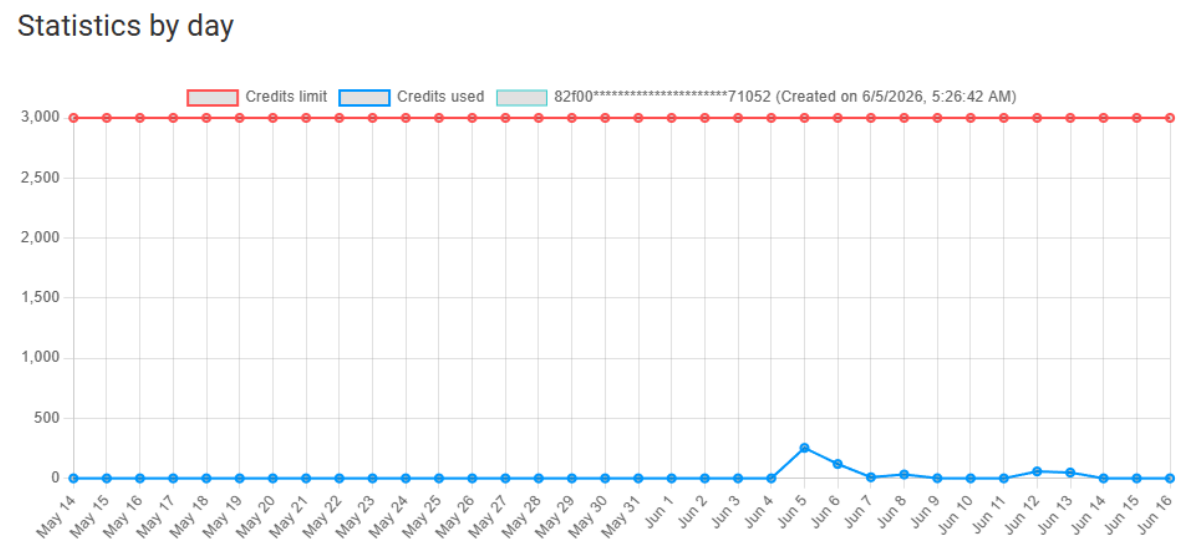


Figure 9: Geoapify API Usage & Limit per Day

This API is somehow reliable as a fallback API because it shares the similar functionality as Foursquare but has a high limit (3000 per day) to handle multiple requests. As in the Figure x provided above, the usage of this API is relatively low from fallbacks, proving that it is suitable to be the backbone of this recommendation system. Besides that, this API is also useful for reverse geocoding, where when the country fails to fetch, it will fallback to Geoapify API to proceed.



```
/**
 * Builds the Geoapify Places URL.
 * Geoapify expects longitude before latitude inside `filter` and `bias`.
 */
Windsurf: Refactor | Explain | ×
function buildPlacesUrl({ latitude, longitude, preference, limit }) {
  const url = new URL(GEOAPIFY_PLACES_URL);
  url.searchParams.set("categories", categoriesForPreference(preference));
  url.searchParams.set(
    "filter",
    `circle:${longitude},${latitude},${GEOAPIFY_RADIUS_METERS}`,
  );
  url.searchParams.set("bias", `proximity:${longitude},${latitude}`);
  url.searchParams.set("limit", limit);
  url.searchParams.set("apiKey", readApiKey());
  return url;
}

/**
 * Builds the reverse geocode url payload.
 */
Windsurf: Refactor | Explain | ×
function buildReverseGeocodeUrl({ latitude, longitude }) {
  const url = new URL(GEOAPIFY_REVERSE_GEOCODING_URL);
  url.searchParams.set("lat", latitude);
  url.searchParams.set("lon", longitude);
  url.searchParams.set("type", "country");
  url.searchParams.set("format", "json");
  url.searchParams.set("lang", "en");
  url.searchParams.set("apiKey", readApiKey());
  return url;
}
```

Figure 10: Configuration of GeoapifyAPI request parameters using URLSearchParams

The buildPlacesUrl() function will be used to construct a Geoapify Places API request based on the latitude, longitude, travel preference and result limit (count). The function maps the user's preference to a suitable Geoapify category and applies a circular filter to restrict the search results within a predefined radius. It also uses a proximity bias so that places closer to the selected coordinates are prioritised. Then, the function will set the result limit and API key before returning the completed request URL. Furthermore, the buildReverseGeocodeUrl() function will be used to construct a reverse geocoding request, which works on converting geographical coordinates into a country name.. Hence, the system identifies the country or location context of the selected coordinates and returns the data in JSON format. Overall, these functions support the Smart Travel Planner system by enabling location-based place recommendations and coordinate-to-country conversion through the Geoapify API.

Recommendation Provider Fallback Mechanism

The project fallback mechanism is one of the key enhancements implemented in this project. The system operates according to the sequence below:

- Attempt to retrieve recommendations data from Foursquare.
- Fallback to Geoapify if Foursquare fails.
- Generate an error response at console when Foursquare fails to return data.

This mechanism is implemented within the fetchRecommendations() function.

```
150       return data;
151     }
152
153     console.error(
154       `Foursquare returned no ${normalizedPreference} results; using fallback recommendations.`,
155     );
156   } catch (error) {
157     console.error(
158       `Foursquare failed for ${normalizedPreference}; using fallback recommendations: ${error.message}`,
159     );
160   }
161
162   const data = await geoapifyService.fetchGeoapifyRecommendations({
163     latitude,
164     longitude,
165     preference: normalizedPreference,
166     limit: normalizedLimit,
167     openAt: normalizeOpenAt(openAt),
168     forceRefresh,
169   }).catch((error) => {
170     console.error(
171       `Geoapify fallback failed for ${normalizedPreference}: ${error.message}`,
172     );
173     return [];
174   });
175   await rememberRecommendations(key, data);
176
177   return data;
178 }
```

Figure 11: Implementation of the fallback function for recommendations

This approach ensures continuous access to recommendation data even if one provider is unavailable. From a software engineering perspective, this design enhances fault tolerance, availability, and user experience.

Weather-based API

Weather information plays a crucial role in travel planning applications, as it influences travellers' decisions regarding planned activities, clothing choices, and the need to modify their itineraries in response to changing weather conditions (Forum, 2024). In the Smart Travel Planner system, a weather module was implemented to provide users with real-time weather information and forecast data for their selected travel destinations.

The OpenWeatherMap API was initially intended to be the main external weather service. OpenWeatherMap is one of the most widely used weather APIs as it provides comprehensive weather information through a simple RESTful interface. It provides functionality for obtaining real-time weather information and future weather forecasts based on geographical coordinates such as latitude and longitude (ReinTech, n.d.).

However, during the implementation phase, the weather service was further improved through the integration of multiple weather providers and additional performance optimisation mechanisms. Instead of relying exclusively on a single external weather provider, the final implementation utilises a fallback mechanism that incorporates multiple services, including Open-Meteo and MET Norway, alongside OpenWeatherMap to improve reliability and data availability. This design improves system reliability by ensuring that weather information remains available even when a provider experiences temporary failures, network disruptions, or rate limitations.

OpenWeatherMap API Integration

The system was connected with OpenWeatherMap to retrieve forecast and current weather data. The weatherService.js file defines and configures two API endpoints.

```
17  const OPENWEATHER_CURRENT_URL =
18    process.env.OPENWEATHER_CURRENT_URL ||
19    "https://api.openweathermap.org/data/2.5/weather";
```

Figure 12: OpenWeatherMap API endpoint configuration for weather services

The Current Weather endpoint is used to retrieve real-time weather conditions, while the Forecast endpoint provides predictive weather data for upcoming time intervals, enabling both immediate and future weather analysis. The application sends requests using the destination's latitude and longitude, enabling weather data to be retrieved for virtually any location worldwide (OpenWeather. n.d.).

For example, when requesting current weather data, the following parameters are included:

```
325  async function fetchOpenWeatherCurrentWeather(latitude, longitude) {
326    const apiKey = readOpenWeatherApiKey();
327    const url = new URL(OPENWEATHER_CURRENT_URL);
328    url.searchParams.set("lat", latitude);
329    url.searchParams.set("lon", longitude);
330    url.searchParams.set("appid", apiKey);
331    url.searchParams.set("units", "metric");
332    url.searchParams.set("lang", "en");
```

Figure 13: Configuration of OpenWeatherMap API request parameters using URLSearchParams

The lat and lon parameters define the destination coordinates, while the appid parameter represents the authentication key provided by OpenWeatherMap (Mozilla Developer, n.d.). The units parameter ensures that temperatures are returned in degrees Celsius, whereas the lang parameter specifies that weather descriptions should be displayed in English.

The system collects useful information from OpenWeatherMap's response and transforms it into a common format that can be utilised across the application. The extracted data includes temperature, feels-like temperature, humidity, wind speed, atmospheric pressure, weather conditions, observation time, city name, and country code. This standardisation process enables the frontend to display weather information consistently, regardless of the data source used.

OpenWeatherMap Forecast Processing

Besides current weather information, OpenWeatherMap is also used to generate weather forecasts. The API returns multiple forecast entries over five days (OpenWeather. n.d.). The entries are processed and organised by their respective dates.

```
20  const OPENWEATHER_FORECAST_URL =
21    process.env.OPENWEATHER_FORECAST_URL ||
22    "https://api.openweathermap.org/data/2.5/forecast";
```

Figure 14: OpenWeatherFORECAST API endpoint configuration for weather services

```
364  async function fetchOpenWeatherDailyForecast(
365    latitude,
366    longitude,
367    startDate,
368    endDate,
369  ) {
370    const apiKey = readOpenWeatherApiKey();
371    const url = new URL(OPENWEATHER_FORECAST_URL);
372    url.searchParams.set("lat", latitude);
373    url.searchParams.set("lon", longitude);
374    url.searchParams.set("appid", apiKey);
375    url.searchParams.set("units", "metric");
376    url.searchParams.set("lang", "en");
```

Figure 15: Configuration of OpenWeatherForecast API request parameters using URLSearchParams

These summaries provide travellers with a clearer understanding of expected weather conditions throughout their trip without overwhelming them with excessive hourly data.

Open-Meteo API Integration

Although OpenWeatherMap provides rich weather information, relying solely on a single provider introduces certain risks. Service outages, API limitations, or temporary failures may prevent users from accessing weather information.

Open-Meteo was incorporated into the weather module to address this issue.

```
16  const OPEN_METEO_URL =
17  "https://api.open-meteo.com/v1/forecast";
```

Figure 16: OpenMETEO API endpoint configuration for weather services

Open-Meteo is a free and publicly accessible weather API that does not require authentication or an API key. It delivers current and forecast weather information based on geographical coordinates (Open-Meteo, n.d.).

In this implementation, Open-Meteo serves as the primary provider in the fallback chain. It retrieves weather variables such as temperature, humidity, apparent temperature, pressure, wind speed, and weather codes. The numerical weather codes returned by Open-Meteo are mapped to human-readable labels, including “Clear Sky”, “Rain”, “Snow”, and “Thunderstorm”, before being displayed to users. The absence of authentication requirements further enhances Open-Meteo’s suitability as a stable and dependable provider under high request loads (GitHub : Open-Metro, n.d.).

MET Norway API Integration

In addition, a third weather data provider, MET Norway’s Locationforecast API, was integrated into the system to enhance reliability and coverage.

```
23  const MET_NORWAY_URL =
24    process.env.MET_NORWAY_URL ||
25    "https://api.met.no/weatherapi/locationforecast/2.0/compact";
```

Figure 17: MET NORWAY API endpoint configuration for weather services

MET Norway is operated by the Norwegian Meteorological Institute and provides free weather forecasts. Users are required to identify their applications through a valid User-Agent header, although it does not require an API key (Open-Meteo[2], n.d.).

```
29  const MET_NORWAY_USER_AGENT =
30    process.env.MET_NORWAY_USER_AGENT ||
31    process.env.NOMINATIM_USER_AGENT ||
32    "SmartTravelPlanner/1.0 (local-development)";
```

Figure 18: MET NORWAY USER AGENT API endpoint configuration for weather services

MET Norway provides detailed forecast time series data, including temperature, humidity, precipitation, wind conditions, and weather symbols. The weather symbols returned by MET Norway are subsequently mapped to standard weather categories used in the Smart Travel Planner system. Integrating MET Norway provides an additional layer of redundancy and reduces the probability of complete weather service disruption.

Weather Provider Fallback Mechanism

The project fallback mechanism is one of the key enhancements implemented in this project. The system operates according to the sequence below:

- Attempt to retrieve weather data from Open-Meteo.
- Fallback to OpenWeatherMap if Open-Meteo fails.
- Generate an error response only when all providers fail to return data.

This mechanism is implemented using the fetchWithFallback() function.

```
92  const data = await fetchWithFallback("current weather", [
93    {
94      provider: PROVIDERS.openMeteo,
95      fetcher: () => fetchOpenMeteoCurrentWeather(latitude, longitude),
96    },
97    {
98      provider: PROVIDERS.openWeather,
99      fetcher: () => fetchOpenWeatherCurrentWeather(latitude, longitude),
100   },
101   {
102     provider: PROVIDERS.metNorway,
103     fetcher: () => fetchMetNorwayCurrentWeather(latitude, longitude),
104   },
105 ]);
```

Figure 19: Implementation of the fetchWithFallback function for retrieving current weather data from multiple providers.

This approach ensures continuous access to weather information even if one provider is unavailable. From a software engineering perspective, this design enhances fault tolerance, availability, and user experience.

Caching and Performance Optimisation

A caching mechanism was implemented to improve performance.

```
33    const CACHE_TTL_MS = 10 * 60 * 1000;
```

Figure 20: Configuration of caching expiration time (TTL) set to 10 minutes

Weather data is cached for ten minutes before expiration. The advantages of this caching approach include:

- Reduced response time
- Fewer external API requests
- Lower risk of exceeding rate limits
- Improved application efficiency

Additionally, timeout handling was implemented to prevent the application from blocking indefinitely due to slow API responses.

```
26    const WEATHER_PROVIDER_TIMEOUT_MS = Number(
27      process.env.WEATHER_PROVIDER_TIMEOUT_MS || 12000,
28    );
```

Figure 21: Configuration of weather provider timeout duration in milliseconds.

Each weather request is automatically terminated after a twelve seconds timeout, allowing the fallback mechanism to activate.

Map/Location-based API

Travel planning applications depend heavily on location tracking, routing, and place discovery since they allow users to view their journey activity plan, determine journey times, and easily navigate overseas locations. In the Smart Travel Planner system, a comprehensive location and mapping module was implemented to provide users with interactive route overviews, nearby place recommendations, and precise coordinate handling (Google[3], n.d.).

The Google Maps Platform, specifically the Google Routes API, was selected as the primary external service for this module. Google APIs are the industry standard for mapping due to their unparalleled geographic accuracy, comprehensive place data, and highly reliable routing algorithms. The system provides functionality for obtaining walking and driving paths, distance metrics, and encoded map polylines based on the user's origin and destination coordinates (Google[3], n.d.).

However, relying strictly on direct, unoptimized API calls to a premium service can introduce issues such as excessive financial costs and rate limiting. During the development stage, the routing architecture was significantly improved to reduce these risks. Instead of a basic data passthrough, the final implementation incorporates server-side polyline decoding, an intelligent credential fallback mechanism, advanced coordinate caching, and cross-platform mobile intents to maximize efficiency and system reliability (Google[3], n.d.).

Google Routes API Integration

The system backend acts as a secure proxy to the Google Routes API to fetch optimized travel paths. The routeService.js file handles the configuration and execution of this endpoint.

```
const GOOGLE_ROUTES_URL =
  process.env.GOOGLE_ROUTES_API_URL ||
  "https://routes.googleapis.com/directions/v2:computeRoutes";
const GOOGLE_ROUTES_FIELD_MASK =
  "routes.duration,routes.distanceMeters,routes.polyline.encodedPolyline";
const response = await fetch(GOOGLE_ROUTES_URL, {
  method: "POST",
  headers: {
    Accept: "application/json",
    "Content-Type": "application/json",
    "X-Goog-Api-Key": readApiKey(),
    "X-Goog-FieldMask": GOOGLE_ROUTES_FIELD_MASK,
  },
```

Figure 22: Google Routes API endpoint and Field Mask header configuration

Unlike standard REST GET requests, the Google Routes API utilizes a POST method. To optimize network bandwidth, the application implements a strict X-Goog-FieldMask header. By explicitly restricting the response payload to only include routes.duration, routes.distanceMeters, and routes.polyline.encodedPolyline, the system avoids downloading unnecessary navigational metadata, drastically improving response times (Google[4], n.d.).

Route Polyline Data Processing

A structured JSON payload containing the origin and destination coordinates as well as the chosen travel mode (such as WALK or DRIVE) is created when requesting route data. The data transformation procedure is a crucial improvement in this stage. (Google[6], n.d.)

```
function decodePolyline(encoded) {
  const points = [];
  let index = 0;
  let latitude = 0;
  let longitude = 0;

  while (index < encoded.length) {
    const latitudeDelta = decodePolylineValue(encoded, () => index++);
    index = latitudeDelta.nextIndex;
    latitude += latitudeDelta.value;

    const longitudeDelta = decodePolylineValue(encoded, () => index++);
    index = longitudeDelta.nextIndex;
    longitude += longitudeDelta.value;

    points.push({
      latitude: latitude / 100000,
      longitude: longitude / 100000,
    });
  }

  return points;
}
```

Figure 23: Implementation of the server-side decodePolyline algorithm

To save bandwidth, Google returns the physical route as an encodedPolyline—a compressed ASCII string. Rather than sending this compressed string directly to the Flutter application, the Node.js backend executes a custom decodePolyline() algorithm. This function uses bitwise operations to translate the ASCII string into an array of readable latitude and longitude coordinates. This architectural decision standardizes the data format and significantly offloads CPU processing from the user's mobile device, allowing the FreeVectorMap UI to render smoothly.

Internal Trip Summary API Aggregation

Through an internal application endpoint, the location and routing data are integrated with other system modules (such as weather and itinerary scheduling) to provide the same front-end user interface.

```
class _SummaryJsonCard extends StatelessWidget {
  const _SummaryJsonCard({required this.json});

  final String json;

  @override
  Windsurf: Refactor | Explain | Generate Function Comment | ×
  Widget build(BuildContext context) {
    return MacPanel(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text('Combined Response Preview (GET /api/trips/:id/summary)',
          const SizedBox(height: 8),
          Container(
            width: double.infinity,
            padding: const EdgeInsets.all(12),
```

Figure 24: Trip Summary API endpoint consumption in the Flutter frontend (GET /api/trips/:id/summary)

This internal API serves as an aggregator for Backend-for-Frontend. For the location module, it supplies the core destination coordinates, nearby Foursquare place recommendations, and agenda route markers within a single JSON response. This design standardizes the data delivery and allows the Flutter application's custom UI components to render complex location data from a single, highly efficient network request.

Client-Side Google Maps Search Intent

While the application provides an excellent in-app overview of the route using vector maps, users navigating real-world environments often require native turn-by-turn voice navigation. The Google Maps Universal Cross-Platform Search API was used in a secondary integration that was directly developed in the Flutter frontend to make this possible.

```
String _googleMapsUrlForAgendaItem(TripAgendaItemModel item) {      Leong, last week • Major UI updates and also
  final latitude = item.latitude;
  final longitude = item.longitude;
  final queryParts = [
    if (item.title.trim().isNotEmpty) item.title.trim(),
    if (item.address.trim().isNotEmpty) item.address.trim(),
    if (latitude != null && longitude != null) '${latitude.toStringAsFixed(6)},${longitude.toStringAsFixed(6)}',
  ];
  final query = queryParts.isEmpty ? 'Google Maps' : queryParts.join(' ');
  return Uri.https('www.google.com', '/maps/search/', {'api': '1', 'query': query}).toString();
}
```

Figure 25: Construction of the Google Maps search URL intent in the Flutter client

In the program, a public URL scheme (https://www.google.com/maps/search/?api=1&query=...) was used to avoid the backend. The program cleverly constructs a search query, giving the destination's title top priority, adding the address, and then returning to precise coordinates. The user's native mapping program opens when they tap the "Open in Google Maps" button. If the device lacks a compatible browser or map app, the system gracefully handles the failure by copying the URL to the device's clipboard and notifying the user via a pop-up SnackBar.

Authentication and Key Fallback Mechanism

Enterprise APIs require strict authentication, and a missing or exhausted API key can lead to a complete breakdown of routing features. To ensure continuous availability, a credential fallback mechanism was built into the backend service.

```
function readApiKey() {      Leong, last week • feat (person 1, 2, 3): implemented
  const apiKey = [
    process.env.GOOGLE_ROUTES_API_KEY,
    process.env.GOOGLE_MAPS_API_KEY,
    process.env.GOOGLE_PLACES_API_KEY,
  ]
    .map((value) => String(value ?? "").trim())
    .find((value) => value.length > 0 && !value.startsWith("your_"));

  if (!apiKey) {
    throw new HttpError(
      503,
      "GOOGLE_ROUTES_API_KEY is not configured for Google Maps Routes API routes.",
    );
  }

  return apiKey;
}
```

Figure 26: Implementation of the readApiKey fallback mechanism for secure authentication

The system executes the following validation sequence before authorizing a request:

- Attempt to load the GOOGLE_ROUTES_API_KEY from the server environment.
- Fallback to the GOOGLE_MAPS_API_KEY when undefined or invalid.
- Use the GOOGLE_PLACES_API_KEY as a final redundancy when still unavailable.

By adding a crucial layer of fault tolerance to the backend architecture, this ensures that routing services remain functional even if a specific cloud credential experiences a configuration error.

Caching and Performance Optimisation

Because external mapping APIs impose strict rate limits and charge per transaction, a highly optimized geospatial caching mechanism was developed to minimize redundant network requests.

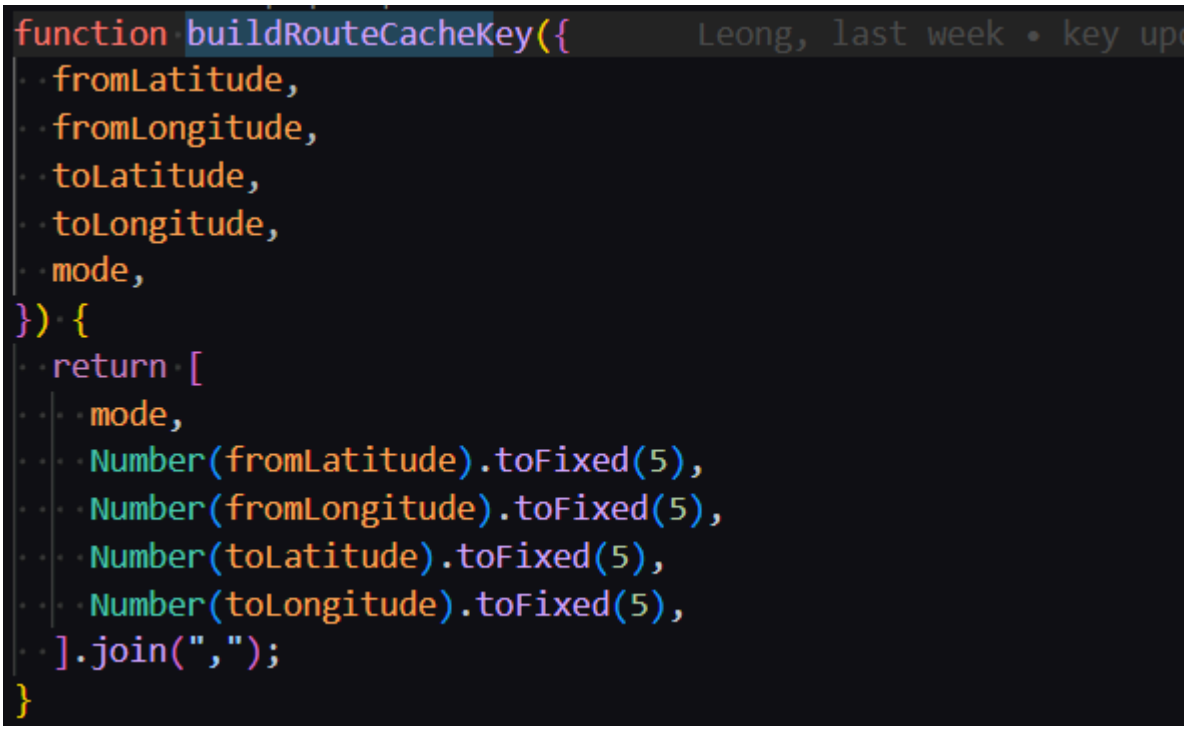


Figure 27: Implementation of the coordinate truncation and caching strategy

Before sending a request to Google, the system generates a unique cache key (buildRouteCacheKey). This key concatenates the travel mode with the origin and destination coordinates, which are mathematically truncated to five decimal places (grouping coordinates within roughly one meter of each other). The backend immediately returns a route if it is in the cache for these coordinates. This caching strategy has several benefits such as decreased API billing expenses, almost immediate response times, and fewer chances of going over rate limitations.

```
const GOOGLE_ROUTES_TIMEOUT_MS = Number(
  process.env.GOOGLE_ROUTES_TIMEOUT_MS || 12000,
);
const response = await fetch(GOOGLE_ROUTES_URL, {
  method: "POST",
  headers: {
    Accept: "application/json",
    "Content-Type": "application/json",
    "X-Goog-Api-Key": readApiKey(),
    "X-Goog-FieldMask": GOOGLE_ROUTES_FIELD_MASK,
  },
  signal: AbortSignal.timeout(GOOGLE_ROUTES_TIMEOUT_MS),
  body: JSON.stringify(
    buildRouteRequest({
      fromLatitude,
      fromLongitude,
      toLatitude,
      toLongitude,
      routeMode,
    }),
  ),
});
```

Figure 28: Configuration of the AbortSignal timeout for the routing service

Furthermore, a strict timeout handler was configured to maintain system stability. An AbortSignal.timeout() set to 12,000 milliseconds wraps each external routing request. The request is terminated if Google's servers fail to respond within 12 seconds. This enables the system to smoothly tolerate network outages without affecting the overall reliability of the service.

Teammate Individual Portfolio

Person 1: Weather Services & Multi-Provider Architecture (Tan Zhi Qi)

Tan Zhi Qi was responsible for designing and implementing the platform's real-time climatology forecasting framework inside src/services/weatherService.js. To ensure continuous service uptime and strictly satisfy the project's robust error-handling requirements, I engineered a resilient, multi-provider failover backend structure. The core system initiates queries targeting the commercial OpenWeather API, authenticated using a protected runtime environment token (OPENWEATHER_API_KEY), to gather standard temperatures and ambient weather conditions for a user's selected destination coordinates. Recognizing the vulnerability of commercial quotas, I programmed automatic fallback logic using keyless, open-access engines. If the primary weather stream hits a rate limit or network exception, the backend smoothly switches to fetch public data from the Open-Meteo API and the MET Norway platform. The MET Norway connection was carefully configured with a custom MET_NORWAY_USER_AGENT string, guaranteeing that the end-user's travel summary dashboard remains populated with live climate details even if primary commercial API credentials fail.

Person 2: Routing Intelligence & Spatial Mapping (Pey Hui Yi)

Pey Hui Yi took structural ownership of the application's geospatial navigation tracking and routing intelligence engines. On the backend, I developed src/services/routeService.js to communicate directly with the Google Routes API, implementing a multi-key exception handler that automatically cycles through GOOGLE_ROUTES_API_KEY, Maps_API_KEY, and GOOGLE_PLACES_API_KEY to safely calculate both driving and walking travel itineraries. Shifting focus to the cross-platform presentation layer, I managed user views within the Flutter mobile framework by writing layout code inside lib/features/trip_summary/view/trip_summary_page.dart. Here, I successfully wired the frontend controls to read latitude and longitude coordinates from the primary database record and translate them into interactive browser deep-links. This setup enables users to tap a location block on their mobile device and instantly redirect to public Google Maps search tools for live, real-world direction updates.

Person 3: Database Administration & Secure Identity Management (Leong Zi Qi)

Leong Zi Qi successfully established the underlying cloud data architecture, user identity platforms, and location profiling systems. I provisioned the remote data tier using a

MongoDB Atlas cluster, constructing the core connection utility within src/db/mongo.js to safely initialize client hooks at runtime via protected MONGO_URI and MONGO_DB_NAME environment tokens. To maximize points within the advanced security grading rubric, I authored src/services/firebaseAuthService.js to seamlessly hook into the Firebase Auth REST Identity Toolkit using a secure FIREBASE_AUTH_API_KEY. This implementation exposed native backend routing endpoints for account creation (signUp), password verification (signInWithPassword), and profile data lookups. Furthermore, I engineered backend service extensions that leverage the Foursquare Places and Geoapify APIs to ingest localized attraction data and reverse-geocoded venue arrays, matching coordinates directly with user travel preferences.

Person 4: Gateway Integration, Geo-Demographics, & System Reliability (Boon Yun Nee)

Boon Yun Nee managed the master gateway integration, open-source mapping visual layers, and global backend failover mechanisms. I authored src/services/countryService.js to process demographic data from the public REST Countries API, mapping text parameter inputs into clean JSON responses containing destination capitals, active currencies, languages, and flag URLs. To protect the backend from commercial mapping downtime, I developed a keyless reverse-geocoding fallback service inside src/services/reverseGeocodeService.js using OpenStreetMap Nominatim and a verified NOMINATIM_USER_AGENT. On the mobile application frontend, Yun Nee configured vector map displays within free_vector_map.dart by utilizing OpenFreeMap tiles and integrating the MapLibre GL CDN toolkit (MapLibre, n.d.). Most importantly, I built the master aggregation controller GET /api/trips/:id/summary inside src/routes/summaryRoutes.js. This essential route queries MongoDB and orchestrates asynchronous background requests across all individual services built by Persons 1, 2, and 3. By wrapping each external network transaction in localized try-catch blocks, Yun Nee ensured a graceful partial-failure design, allowing the traveler to view their primary notes and country info even if a teammate's external API experiences a timeout.

Security & Centralized Error Handling

Security is integrated directly into both the network level and runtime layers of an application. For example, credentials and secret API tokens are fully isolated from the version control system by using structured environment variables that match a documented template. On the client app layer managed by Flutter (Google, n.d.-a), requests automatically retrieve saved user validation tokens from Firebase (Google, n.d.-b) and inject them as an Authorization Bearer Header (Bearer) into outgoing HTTP requests. This makes sure that any trip actions (/api/trips) are very tightly blocked off from unauthorized external actors.

To achieve robust error handling, the master summary endpoint wraps each external service connection inside individual safety containers. If an external API encounters a rate-limit error, network drop, or invalid key, the centralized error handler intercepts the error stack, appends an empty array or warning log to that specific node, and safely allows the rest of the application response to load normally without crashing.

JSON Web Token (JWT) is an open standard (RFC 7519) that specifies a concise and self-sufficient method for securely sending information between parties in the form of a JSON object. (jwt.io, n.d.) This data can be confirmed and relied upon as it is digitally signed. JWTs may be signed with a secret key (utilizing the HMAC algorithm) or through a public/private key pair via RSA or ECDSA. It is usually used in authentication or information exchanging. (jwt.io, n.d.) JWT has a header, payload and signature structure that are separated by dots (.). (jwt.io, n.d.)

```
55  function generateToken(user) {
56    return jwt.sign({ id: user.id, email: user.email }, readJwtSecret(), {
57      algorithm: JWT_ALGORITHM,
58      subject: String(user.id),
59      expiresIn: JWT_EXPIRES_IN,
60      ...optionalJwtClaims(),
61    });
62  }
63
```

Figure 29: Function to generate a JSON web token

The figure above shows the implementation of JWT in the Smart Travel Planner system, hence when a user successfully logs in with their user credentials, a JWT will be returned that will expire in 7 days.

```
18    const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN || "7d";
19    const JWT_ALGORITHM = "HS256";
```

Figure 30: JWT expiration limit and algorithm used

The figure shows that the JWT will expire 7 days after the creation, and it is using the HS256 algorithm. This code can be found in the authService.js, this file centralizes all of the security helpers to hash passwords, comparing passwords, JWT token generation and verification.

Lastly, the backend aggregation layer also works to protect the system in several ways:

- API keys always remain in backend environment variables
- Endpoints require JWT authentication
- CORS enforced centrally
- Repository lookup includes ‘ownerUserId’, prevents cross-user trip access

Testing Framework & Test Results

Our group followed a rigorous verification process using an integrated Postman Documentation Collection stored directly inside the code repository (postman/SmartTravelPlanner.postman_collection.json) to execute sandbox queries (Postman, n.d.). This collection contains pre-configured runtime templates for every auth, trip, proxy, and summary endpoint.

To ensure continuous quality and secure our documentation, we implemented modern DevOps / CI/CD Automation. Our GitHub repository uses automated testing workflows (firebase-hosting-pull-request.yml). Every time a team member pushes a change or opens a pull request on their feature branch, GitHub Actions fires off an automated pipeline to check code consistency, verify secrets (GITHUB_TOKEN), and test deployment health before allowing it to be merged into the main development branch (GitHub, n.d.).

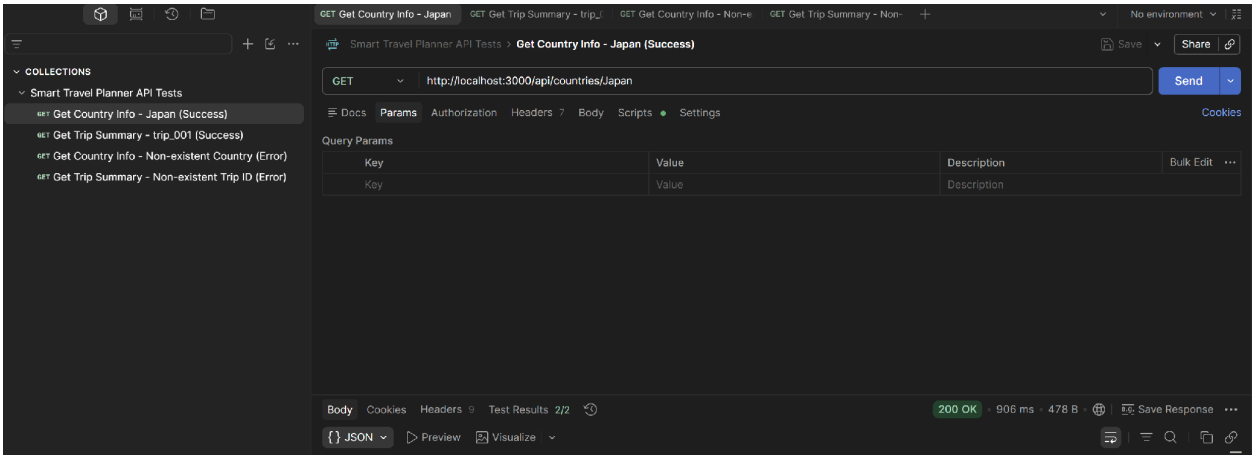


Figure 31: Postman execution test for the /api/countries/:name endpoint.

Challenges, Limitations & AI Reflection

- **Asynchronous Integration Complexity:** Coordinating and formatting structures across five distinct external APIs (OpenWeather, Google Routes, Foursquare, Geoapify, and REST Countries) required careful design to prevent slow response times and race conditions (Current weather data, n.d.; Florez, F. n.d.; Foursquare, n.d.; Geoapify, n.d.; Google for Developers, n.d.-a).
- **API Rate Limits & Fallbacks:** Overcoming strict usage caps on commercial providers by building open-access failover services using Open-Meteo (Open-Meteo, n.d.), MET Norway (Norwegian Meteorological Institute, n.d.), and OpenStreetMap Nominatim (OpenStreetMap, n.d.) was a major technical hurdle that greatly enhanced the application's resilience.
- **AI Disclosure:** AI tools were used exclusively as an architectural design companion to help map out clean routing structures and organize the technical text sections of this engineering portfolio.

Links

- GitHub: https://github.com/leong308/api_6005cmd.git
- Live-hosted Web: https://smart-trip-planner.web.app/
  (backend with long inactivity will made it hibernate, any request to backend will wake it up for around 1 minute, please retry a moment after if first request fails)
- Firebase: https://console.firebase.google.com/u/1/project/cmdapi-22ead
  (invited shahriman.said@newinti.edu.my as Viewer Role)
- Youtube: https://www.youtube.com/watch?v=qmEVuif34Y0

References

Current weather data. (n.d.). https://openweathermap.org/api/current

Florez, F. (n.d.). *REST Countries - Get information about countries via a RESTful API*. Retrieved from https://restcountries.com/

Foursquare. (n.d.). *Place Search API Reference*. Foursquare Developer. Retrieved from https://docs.foursquare.com/fsq-developers-places/reference/place-search

Forum, T. (2024). Do Weather Changes Affect Travel Weather Planning Decisions? Retrieved from https://www.travelforumworld.com/travel-weather-planning/do-weather-changes-affect-decisions/

Geoapify. (n.d.). *Geocoding API Documentation*. Retrieved from https://www.geoapify.com/geocoding-api

GitHub. (n.d.). *GitHub Actions Documentation*. Retrieved from https://docs.github.com/en/actions

GitHub[2] , (n.d.). Add icons to graphically represent each weather_code. Retrieve from https://github.com/open-meteo/open-meteo/issues/789

Google. (n.d.). *Flutter Documentation*. Retrieved from https://flutter.dev/docs

Google[2]. (n.d.). *Firebase Authentication REST API*. Retrieved from https://firebase.google.com/docs/reference/rest/auth

Google[3]. (n.d.) *Get a route | Routes API*. Google for Developers. Retrieved from: https://developers.google.com/maps/documentation/routes/compute_route_directions

Google[4]. (n.d.) *Choose fields to return | Routes API*. Google for Developers. Retrieved from: https://developers.google.com/maps/documentation/routes/choose_fields

Google[5]. for Developers. (n.d.-a). *Routes API Documentation*. Google Maps Platform. Retrieved from https://developers.google.com/maps/documentation/routes

Google[6]. (n.d.) *Encoded Polyline Algorithm Format*. Google for Developers. Available at: https://developers.google.com/maps/documentation/utilities/polylinealgorithm

jwt.io. (n.d.). Introduction to JSON Web Tokens. Retrieved from https://www.jwt.io/introduction#what-is-json-web-token-structure

MapLibre. (n.d.). *MapLibre GL JS Documentation*. Retrieved from https://maplibre.org/maplibre-gl-js/docs/

MongoDB. (n.d.). *MongoDB Atlas Documentation*. Retrieved from https://www.mongodb.com/docs/atlas/

Mozilla Developer, (n.d.). URLSearchParams: set() method. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams/set

Node.js Foundation. (n.d.). *Node.js Documentation*. Retrieved from https://nodejs.org/en/docs/

Norwegian Meteorological Institute. (n.d.). *MET Norway Weather API*. Retrieved from https://api.met.no/

Open-Meteo. (n.d.). *Open-Meteo API Documentation*. Retrieved from https://open-meteo.com/en/docs

Open-Meteo[2]. (n.d.). Met Norway API. Retrieved from https://open-meteo.com/en/docs/metno-api

OpenStreetMap. (n.d.). *Nominatim Usage Policy*. OpenStreetMap Foundation. Retrieved from https://operations.osmfoundation.org/policies/nominatim/

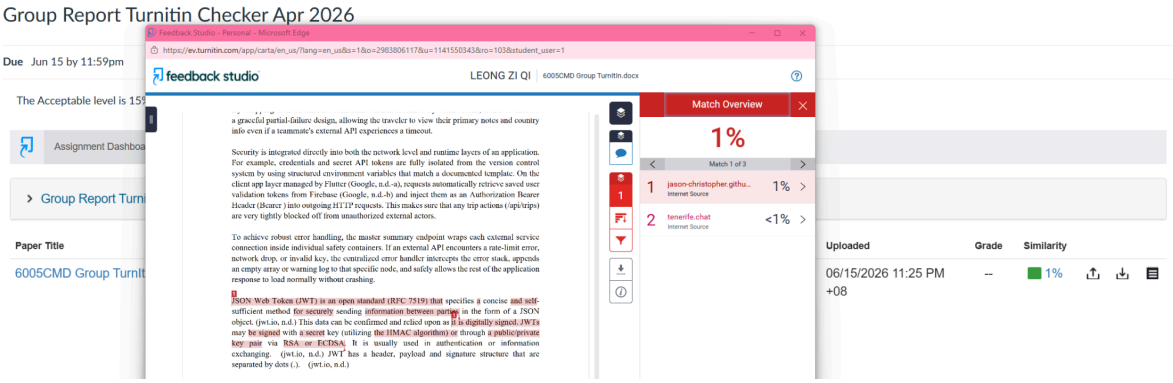
OpenWeather. (n.d.). OpenWeather API Documentation. Retrieved from https://openweathermap.org/api/one-call-4?collection=one_call_api

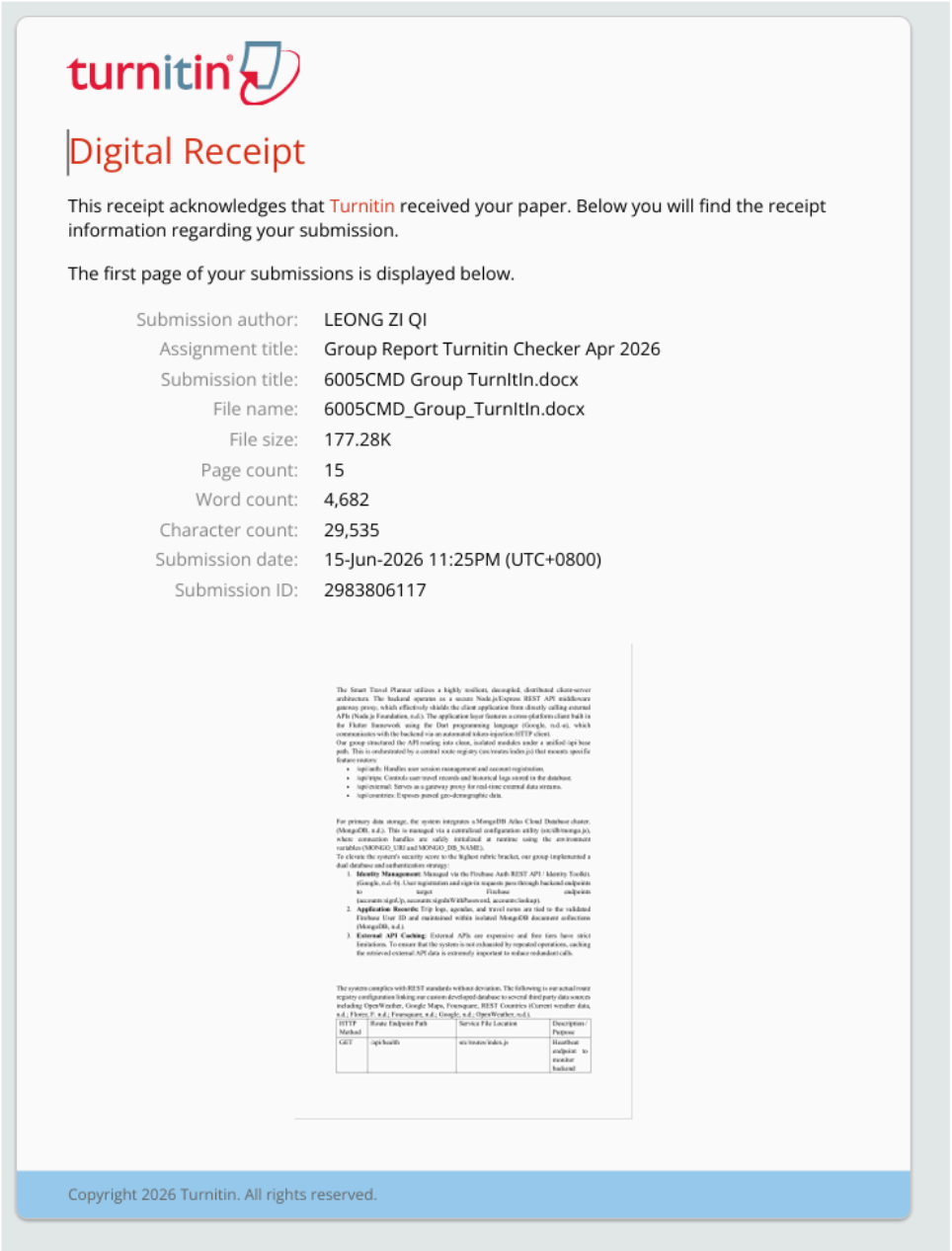
Postman. (n.d.). *Postman API Platform*. Retrieved from https://www.postman.com/

ReinTech, (n.d.). OpenWeatherMap API. Retrieved from: https://reintech.io/term/openweathermap-api-guide

Smith, G. (n.d.). Foursquare API Offers Possibilities To Engage with Customers. Retrieved from: https://www.ignitesocialmedia.com/foursquare-marketing/foursquare-api-offers-possibilities-to-engage-with-customers/

Appendix





Words count (excluding image subtitle): **4682**